

Question 1 Answer:

Generic sorting algorithms are reusable, type-safe, and avoid boxing/unboxing, unlike non-generic ones that require duplication or casting.

Question 2 Answer:

Lambda expressions make sorting concise, readable, and flexible by embedding comparison logic inline.

Question 3 Answer:

Dynamic comparers allow sorting across multiple data types without rewriting logic for each type.

Question 4 Answer:

Implementing `IComparable<T>` lets derived classes define custom sorting rules directly within the class.

Question 5 Answer:

`Func<T, TResult>` simplifies delegate creation, making generic programming cleaner and more expressive.

Question 6 Answer:

Anonymous functions provide inline logic but can be less concise than lambdas; lambdas are generally more readable and efficient.

Question 7 Answer:

Generic methods like `Swap` eliminate duplication by working for all types with type safety.

Question 8 Answer:

Multi-criteria sorting is complex to implement but provides precise ordering and flexibility for real-world needs.

Question 9 Answer:

`default(T)` ensures safe initialization in generics, returning 0 for value types and null for reference types.

Question 10 Answer:

Constraints enforce type requirements, preventing misuse and increasing method reliability.

Question 11 Answer:

Delegates for string transformations enable functional, reusable, and modular code design.

Question 12 Answer:

Delegates allow flexible implementation of math operations while reusing the same processing logic.

Question 13 Answer:

Generic delegates transform data structures generically, improving flexibility and reducing code repetition.

Question 14 Answer:

`Func` simplifies delegate declaration, improving readability and reducing boilerplate.

Question 15 Answer:

Action is preferred for operations that perform work but don't return results.

Question 16 Answer:

Predicates enhance clarity by encapsulating conditions into reusable boolean logic.

Question 17 Answer:

Anonymous functions localize logic, improving modularity and customization without extra method definitions.

Question 18 Answer:

Use anonymous functions when logic is simple, one-time, and doesn't need reuse.

Question 19 Answer:

Lambdas are essential for concise, inline, and expressive coding in modern C#.

Question 20 Answer:

Lambdas make math operations more expressive by embedding formulas directly in the code.